

The gradient in the single-site rate sampler

What it is, what it costs, how good it is,
and the three things it cannot do

`my_extendedSampling` / `softmax`

2026-10-05

Abstract

Every move of `classes/rate_sampler.py` is steered by one gradient of the energy with respect to a site's logits. This note collects everything established about that gradient: how it is defined and computed, the two variants that exist and why the more faithful one was chosen over the one that sounds cheaper, its measured cost and memory (both exactly flat in the number of relaxed sites), and its measured quality — which is modest, strongly dependent on where in sequence space it is evaluated, and essentially useless in magnitude while being genuinely useful in rank. It also records three negative results that bound what the gradient can be asked to do: its first-order prediction is wrong by a factor of $\sim 10^3$, a single substitution perturbs the attention map almost everywhere rather than locally (so single-site reasoning cannot be composed), and the backward pass appears not to be bit-reproducible, making a specific reported substitution one draw rather than a fixed property.

Contents

1	Scope, and where every number comes from	2
2	What the gradient is, and where it enters	2
2.1	Definition	2
2.2	Where it enters	2
2.3	A structural caveat on the gauge	3
3	Two ways to compute it, and why the production one was chosen	3
3.1	The two variants	3
3.2	Why the single-site variant, and a correction	3
3.3	Do the two agree?	4
4	Cost: flat in the number of relaxed sites	4
4.1	The measurement	4
4.2	Why it must be flat	4
4.3	The corollary that drives the design question	5
5	Quality: how good is the gradient's choice?	5
5.1	The measurement	5
5.2	What this says	5
5.3	A historical fix worth not re-learning	6
6	Three things the gradient cannot do	6
6.1	It cannot predict ΔU (first-order failure)	6
6.2	A single substitution is not a local perturbation	6
6.3	The gradient may not be reproducible	7

7	Summary table	8
8	Open items	8

1 Scope, and where every number comes from

All figures below are measured and committed. Sources:

- `classes/rate_sampler.py` — `_grad_pass_site`, the production gradient.
- `customs/custom_esm/models/esm3.py` — `predict_attention` / `_last_attention`.
- `tests/test_attention_locality.py` → `zero_polymer/attention_locality.txt` — locality, cost vs. k , method equivalence, reproducibility.
- `tests/test_steepest_descent.py` → `protein_g/steepest_descent_CONTROL.txt`, `zero_polymer/steepest_descent_results.txt`, `zero_polymer/steepest_descent_S30_M51.txt` — pick quality.
- `tests/compare_grad_methods.py` → `zero_polymer/compare_grad_results.txt` — the two variants.
- `tests/scan_U_am_landscape.py` → `*/scan_U_am*.txt` — energy scale, numerical floor.
- `tests/test_joint_coupling_N.py` → `*/test_joint_coupling_N_results.txt` — first-order quality.
- `softmax/DEVLOG.txt` — chronology and the decisions below.

Two references recur: `protein_g` ($L = 56$) and `zero_polymer` ($L = 566$). $K = 25$ is the width of `SEQUENCE_USED_VOCAB` (20 standard residues plus X, B, U, Z, 0); the proposal targets only the 20 standard ones minus the current residue, so 19 competitors.

2 What the gradient is, and where it enters

2.1 Definition

A sequence is held as per-site logits $x \in \mathbb{R}^{L \times K}$, mean-centred along the class axis to gauge-fix the softmax translation invariance. The gradient the sampler uses is

$$\nabla U = \frac{\partial U}{\partial x_s} \in \mathbb{R}^K, \quad U = \lambda_{\text{am}} U_{\text{am}} + \lambda_S S, \quad U_{\text{am}} = \sum_{i < j} \left(\log a_{ij} - \log a_{ij}^{\text{ref}} \right)^2, \quad (1)$$

where a is the attention map: the head-averaged attention of ESM3's last block, BOS/EOS stripped and symmetrised, obtained from a differentiable forward pass on soft sequence probabilities.

2.2 Where it enters

The gradient sets the *drift* of a Gaussian displacement — one HMC leapfrog half-step with fresh momentum, equivalently one MALA step:

$$\delta x \sim \mathcal{N}(\mu, \sigma^2 I), \quad \mu = -\frac{\Delta t^2}{2M} \nabla U, \quad \sigma = \Delta t \sqrt{T/M}. \quad (2)$$

The proposal is $\arg \max_j \delta x_j$ over the 19 competitors. Two consequences matter here and are developed in `informedness.tex`:

1. **Only the gradient’s *ranking* is ever consumed.** Since $\Delta t^2/(2M) > 0$ is a scalar, $\arg \max_j \mu_j = \arg \min_j (\nabla U)_j$ regardless of Δt , T , M . The gradient’s *magnitude* never reaches the proposal except through the ratio μ/σ , which sets how reliably the realised draw follows that ranking.
2. **Therefore gradient quality means rank quality.** Everything in §5 measures whether the gradient’s preferred substitution is a good one, not whether it predicts ΔU .

2.3 A structural caveat on the gauge

∇U is $\partial U/\partial(\text{logits})$, not $\partial U/\partial(\text{probs})$. Because softmax is translation invariant, this derivative sums to exactly zero across the K classes at every site. The current class’s component is therefore an aggregate (the negative sum of all others), not a component on the same footing as any single competitor — so comparing ∇U_{cur} against ∇U_j is not the clean two-point comparison it appears to be. This mattered for the abandoned joint design, which included “stay” in the competition. It does *not* affect the current sampler, which excludes the current residue from the competition entirely and so only ever compares competitor components against each other. Recorded because it would resurface immediately if “stay” were ever reintroduced as a candidate.

3 Two ways to compute it, and why the production one was chosen

3.1 The two variants

Whole-sequence relaxed (`whole_sequence_grad`). Every site is relaxed through $\text{softmax}(x/T_{\text{sftm}})$ at once; one backward pass yields the full (L, K) gradient. This is what the original (removed) joint design used. It survives only as a diagnostic reimplementations in `tests/compare_grad_methods.py`, `tests/test_joint_coupling_N.py` and `tests/test_multisite_tradeoff.py`.

Single-site relaxed (`_grad_pass_site`). Only the chosen site is relaxed; every other site is held at its *exact one-hot* value:

```
hard = eprot.get_probs()           # (L,K) one-hot, NO grad
site_logits = eprot.logits[site].detach().clone().requires_grad_(True)
soft_row = torch.softmax(site_logits/pars['T_sftm'], -1).unsqueeze(0)
probs = torch.cat([hard[:site], soft_row, hard[site+1:]], dim=0)
                                # ^ a FULL (L,K) tensor
am = self.model.predict_attention(sequence_probs=probs)
U = pars['lambda_am']*compute_U_am(am, self.ref_eprot.am) + ...
U.backward()
grad = site_logits.grad           # (K,) -- only this leaf differs
```

This is the production path, adopted at the 2026–09–19 pivot to single-site moves.

3.2 Why the single-site variant, and a correction

Not because it is cheaper. It is not cheaper (§4), and treating it as the “light” option is a natural but incorrect reading of the code. It was chosen because it is a more *faithful* partial derivative: every other site stays at the exact discrete value used by the accept/reject step and the reverse-probability pass, instead of being blurred by T_{sftm} . The whole-sequence variant evaluates the derivative at a point that is not the state the sampler is actually in.

3.3 Do the two agree?

Yes, to the precision that matters. On `zero_polymer`:

source	sites	cosine	$\max g_1 - g_L $	same choice?
<code>compare_grad_results.txt</code>	11	0.9999–1.0000	—	10/11
<code>attention_locality.txt</code> §3	3	0.999962–0.999997	7.7×10^{-4} – 7.7×10^{-2}	3/3

Table 1: The two gradient variants against each other. “Same choice” is the only thing the sampler consumes: the preferred substitution among the 19 competitors. Note they are *not required* to be identical — they evaluate the derivative at different points — so agreement is a finding, not a tautology.

One site (43) was flagged: cosine = 1.0000 yet the two methods picked *differently* (site-only $\rightarrow$ W, $\Delta U = +680.10$; whole-sequence $\rightarrow$ A, $\Delta U = -127.26$). Near-parallel gradients with different $\arg \max$ means the top two competitors were nearly tied, so an arbitrarily small magnitude difference flips the winner. This is the same fragility that §6.3 returns to, and it is a property of $\arg \max$ on a near-degenerate field, not a disagreement between the methods.

4 Cost: flat in the number of relaxed sites

4.1 The measurement

`tests/test_attention_locality.py` §2 times a gradient with k rows relaxed and $L - k$ held one-hot. $k = 1$ is exactly `_grad_pass_site`; $k = L$ is exactly `whole_sequence_grad`; nothing else varies.

k	t_{fwd} (ms)	$t_{\text{fwd+bwd}}$ (ms)	backward (ms)	peak mem (MB)
1	61.51	151.03	89.53	6369.6
2	61.53	151.14	89.62	6369.6
5	61.29	151.18	89.89	6369.6
20	61.45	150.91	89.45	6369.6
100	61.80	150.93	89.12	6369.7
566	61.71	151.02	89.31	6369.8

Table 2: `zero_polymer`, $L = 566$. Ratio $k=1$ vs. $k=L$: 1.0002. Memory differs by 0.2 MB out of 6.4 GB (0.003%).

4.2 Why it must be flat

The intuition that one row should be cheaper fails on the chain rule, not on an implementation detail. To obtain $\partial U / \partial x_s$ for a *single* site you still need the gradient at every position of every layer, because self-attention makes layer $k+1$ at *every* position depend on layer k at position s . The (L, d_{model}) activation gradient is propagated through all 48 layers either way. Only the final projection differs: onto one row of the (L, K) input instead of all L .

With ESM3-open ($L = 566$, $d_{\text{model}} = 1536$, 48 layers, $K = 25$) and model parameters frozen (`requires_grad=False` in `ESM3.__init__`), so only input gradients are computed:

transformer backward ≈ 816 GFLOP	identical for every k
embedding leaf, all L rows ≈ 0.0217 GFLOP	
embedding leaf, one row ≈ 0.00004 GFLOP	

so the largest possible saving from doing one row instead of L is $\sim 0.001\%$ of a pass. The measured spread in Table 2 is 0.02% — i.e. the predicted effect is about $100\times$ *below* the run-to-run noise, which is exactly why the column reads flat. Memory is flat for the same reason: the same activations must be stored to differentiate through attention regardless of how many input rows carry a gradient.

4.3 The corollary that drives the design question

A gradient covering *all* sites costs what a gradient covering *one* site costs. An N -site move therefore gets its N mutations for free.

Per-move cost is $2\times$ gradient + $1\times$ exact energy ≈ 360 ms either way (measured: single-site 360.75 ms, N -site 359.56 ms, ratio 0.997). The cost argument favours multi-site moves unconditionally; only *acceptance* can argue against them. No cheap win exists on the other side either: making the single-site gradient genuinely cheaper would require truncating the backward to a window around the site, which is an approximation and would break the exactness the proposal relies on.

5 Quality: how good is the gradient’s choice?

5.1 The measurement

For each tested site, brute-force the true energy of all 19 candidate substitutions and ask (i) is the gradient’s pick the true optimum (top-1), and (ii) how does the true ΔU at the gradient’s pick compare to the average over the other candidates.

run	top-1	$\overline{\Delta U}$ at pick	at others	advantage	adv./scale
protein_g, MUTS=5 (Hd 8.9%)	2/10	+17.20	+20.99	3.79	12.0%
zero_polymer, MUTS=5 (Hd 0.9%)	2/10	+474.38	+542.46	68.08	11.3%
zero_polymer, MUTS=51 (Hd 8.8%)	10/30	−360.03	+244.25	604.28	—

Table 3: “scale” is each reference’s own mean ΔU for a single mutation from the reference (31.44 and 603.35 respectively), the only basis on which two different energy scales can be compared.

5.2 What this says

The gradient is weakly but genuinely informative. Top-1 of 2/10 means it finds the true optimum rarely. But its pick is consistently better than a random legal candidate, and — the point that makes the two references comparable — by nearly the *same fraction* of each protein’s energy scale: 12.0% vs. 11.3%. Gradient quality does not degrade with sequence length.

Where you measure dominates what you measure. The third row is the same gradient on the same protein, evaluated away from U_{am} ’s floor instead of at it. Top-1 rises $20\% \rightarrow 33\%$, the fraction of picks that are downhill at all rises $40\% \rightarrow 73\%$, and the mean ΔU at the pick flips sign, from +474 to −360. A measurement taken at MUTS= 5 on a 566-residue reference (0.9% drift) is a measurement at the energy floor, where almost every move is uphill, and it understates the gradient badly. This invalidated an earlier conclusion of ours about proline (see §6) and it is the single most important caveat when reading any older diagnostic in this project.

The descent-direction check always passes. With the momentum term removed, $\mu_A \cdot \nabla U \leq 0$ at every tested site in every run. This is true by construction, so it is a bug detector (sign error, wrong gauge, mis-indexed vocabulary), not a result — and it has never fired since the non-canonical fix below.

5.3 A historical fix worth not re-learning

The very first real runs found the gradient pointing at **X** (and potentially **B,U,Z,0**) — the ambiguity codes at the tail of **SEQUENCE_USED_VOCAB** — even when doing so *increased* the true energy. These are now excluded from the competition by *slicing* them out of the candidate tensor rather than by adding a large negative penalty: a finite penalty could in principle be approached by Δt^2 -scaled gradients at large Δt , whereas a sliced-out class is structurally absent. Note the asymmetry this leaves: **mutate()**, used to build test starting points, still draws from all 25 letters, so test sequences can *contain* residues the sampler could never *propose* (MUTS= 51 produced 12 such residues).

6 Three things the gradient cannot do

6.1 It cannot predict ΔU (first-order failure)

At $N = 1$ the proposed move *is* the gradient’s own greedy pick, and **sum_linear** is its first-order prediction of the resulting energy change. Against the truth:

	$ \text{sum_linear} $	$ \Delta U $	ratio	sign agrees
protein_g	0.0480	57.73	8.3×10^{-4}	13/20
zero_polymer	0.8735	458.29	1.9×10^{-3}	14/20

The linear prediction is three orders of magnitude too small, and its *sign* is right barely more often than chance. Worse, on **zero_polymer** three of the four largest- $|\Delta U|$ cases have the sign *wrong*: predicted -2.56 , -1.01 , -0.23 where the truth was $+2246.9$, $+1819.7$, $+950.9$.

This is not a defect to fix but a boundary to respect. A one-hot substitution is a finite jump across a vocabulary simplex, not an infinitesimal displacement, and U_{am} runs through 48 nonlinear attention layers. The sampler never relies on this: it uses only the ranking (§2), and the Metropolis test uses the *true recomputed* energy. Any future design that tries to use ∇U to *estimate* ΔU — for instance to skip the exact energy evaluation, or to pre-screen candidates — is contradicted by this table.

6.2 A single substitution is not a local perturbation

The natural assumption is that changing site s changes the attention map in row/column s . It does not. Splitting $|\delta a| = |a(\text{seq}_B) - a(\text{seq}_A)|$ into LOCAL (row or column s ; $2L - 1 = 1131$ entries) and DISTAL (the other 319,225):

site	DISTAL share	per-entry L/D	ΔU_{am}	from LOCAL	from DISTAL
80	97.4%	$7.5\times$	-257.65	$+51.15$ (−19.9%)	-308.80 (119.9%)
283	97.7%	$6.6\times$	-180.14	$+4.51$ (−2.5%)	-184.65 (102.5%)
471	97.9%	$6.0\times$	$+573.01$	$+10.86$ (1.9%)	$+562.15$ (98.1%)

Table 4: **zero_polymer**. Only 565 of the 159,895 summed pairs involve site s at all — 0.353% of the energy terms — so strict locality would cap what one mutation could do.

Both halves of this are true and the second is what matters: *per entry* the perturbation *is* concentrated at the mutated site, 6.0–7.5 $\times$ stronger in LOCAL. But DISTAL has 282 $\times$ more

entries, so it carries $\sim 98\%$ of the total and 98–120% of the energy change. At site 80 the two halves even have *opposite signs*: pairs touching the mutated site push U_{am} *up* by 51 while the rest pulls it *down* by 309. The net change is not merely dominated by distal pairs — it is produced by them, against the local ones.

The decay profile confirms real but shallow concentration, and notably does not fall to zero. At site 471: 2.31×10^{-4} at distance 0, decaying to 1.02×10^{-5} at distance 51–100 — then *rising* to 2.46×10^{-5} beyond 100. Every tested site shows this upturn in the farthest bin. Sequence-distant is not distant in 3D, so this is what a structural contact would look like; unproven here, and worth a look only if the question becomes load-bearing.

Consequence. This is the same phenomenon as the measured cross-site coupling, seen from the map side instead of the energy side. It is why single-site measurements cannot be composed into multi-site predictions, and therefore why the multi-site question needed its own experiments rather than an argument from single-site data.

6.3 The gradient may not be reproducible

Status: CONFIRMED. A re-run of `tests/test_multisite_tradeoff.py` with every RNG seeded identically reproduced 68 of 84 cells but not the other 16. Investigation:

- starting points were *bit-identical* (same Hd, same U_A to the last decimal at all six), so the forward pass is reproducible — consistent with the independent noise-floor check, which returns *exactly* 0 for two `predict_attention` calls on the same sequence, on both references;
- the changed cells rose monotonically with N : 1 at $N=2$, 3 at $N=5$, 3 at $N=8$, 4 at $N=12$, 5 at $N=20$;
- the run used a commit in which site selection and every seed formula were unchanged.

`tests/test_attention_locality.py` §4 measured it directly — two whole-sequence gradients on one input:

bit-identical?	No
$\max g_1 - g_2 $	7.87×10^{-2} against $\ g\ \approx 43.9$, i.e. 0.18% relative
sites changing preferred substitution	$3/566 = 0.53\%$

Why the perturbation is this large. 0.18% is far above the $\sim 10^{-7}$ one would expect from reduction-order effects in fp32. The backward runs under `torch.autocast(bfloat16)`, and bfloat16 carries ~ 8 mantissa bits, i.e. $\sim 0.4\%$ relative precision. A change in summation order therefore lands at exactly this scale. The forward stays bit-reproducible because it executes the same kernels in the same order every time; the backward does not, because its reductions use atomics.

Does the flip rate explain the re-run? Yes, in both shape and magnitude. With $p = 0.0053$, a trial touching N sites is affected with probability $q = 1 - (1 - p)^N$, and a cell of 10 trials contains at least one affected trial with probability $1 - (1 - q)^{10}$:

N	q (per trial)	$P(\geq 1 \text{ of } 10 \text{ trials})$	observed cells moved
2	1.06%	10.1%	8%
5	2.62%	23.3%	25%
8	4.16%	34.6%	25%
12	6.18%	47.2%	33%
20	10.08%	65.5%	42%

The observed column counts cells whose *median* moved, which must sit below “at least one trial moved” — a median over 10 values is robust, so a cell can contain a changed trial without its median shifting. It does, at every N , and the two rise together. A flip rate measured independently, on a different test, accounts for the discrepancy quantitatively.

What follows. Not a correctness problem. Metropolis–Hastings needs only the proposal probability actually used, and the Boltzmann validation passed on its own terms. What follows is interpretive, and applies to every diagnostic in this project:

- “the gradient prefers X at site s ” is *one draw*, not a fixed property, wherever two competitors are nearly tied — which is exactly the site-43 flag in §3, now explained rather than merely noted;
- a gradient-based test is *not* expected to re-run identically, and a diff between two runs is not a valid consistency check. (We offered exactly that advice earlier; it was wrong and is withdrawn.)
- single-site results quoted at one specific site — the `protein_g` site-4 outlier, for instance — should be read as stable only in so far as they have reappeared across runs, which that one has since 2026–09–21.

7 Summary table

Question	Answer, and where it is established
Is the single-site gradient cheaper than the whole-sequence one?	No. Flat in k : 151.03 vs. 151.02 ms, ratio 1.0002 (Table 2).
Does it use less memory?	No. 6369.6 vs. 6369.8 MB, a 0.003% difference.
Why use it then?	Fidelity: other sites stay at their exact one-hot values, matching the context the accept/reject uses.
Do the two variants agree?	Yes — cosine ≥ 0.99996 , same choice at every site tested bar one near-degenerate case.
Is the gradient’s pick any good?	Weakly, consistently: 11–12% of the energy scale better than a random candidate, at both lengths. Top-1 20–33%.
Does quality depend on where it is evaluated?	Strongly. At the floor the mean pick is +474; in the bulk, –360.
Can it predict ΔU ?	No — off by $\sim 10^3$, sign right $\sim 65\%$ of the time.
Is a single substitution local?	No. 98% of the energy change comes from pairs not involving the mutated site (Table 4).
Is it reproducible?	Forward yes (noise floor exactly 0); backward no — 0.18% relative under <code>bfloat16</code> autocast, flipping the preferred substitution at 0.53% of sites (§6.3).
What does this imply for multi-site moves?	Cost favours them unconditionally (N mutations per move at single-site price); only acceptance can argue against.

8 Open items

1. **Backward reproducibility** — resolved (§6.3): confirmed non-reproducible, quantitatively consistent with the observed re-run discrepancy. No action proposed: forcing deterministic kernels would cost speed for an effect that does not threaten correctness, and the flip rate is a useful thing to know rather than a defect to remove.

2. **Gradient quality in the bulk is based on one run** (30 sites, MUTS= 51, one seed). The 12%-of-scale figure that makes the two references comparable comes from the *floor* runs; the bulk equivalent has no `protein_g` counterpart yet. `REF=protein_g SITES=30 MUTS=20` would supply it.
3. **The far-field upturn** in the decay profile (Table 4 discussion) is unexplained; a 3D-contact check against decoded structure would settle it.
4. **Nothing here constrains $U_{\text{structure_ce}}$** , whose gradient magnitudes were measured 100–1000× smaller and whose branch is paused (see `DEVLOG_U_ce_backup.txt`).